

CoderCommands MCP System

Step 04 Record — Sync Service (Watcher + Transactional Write)

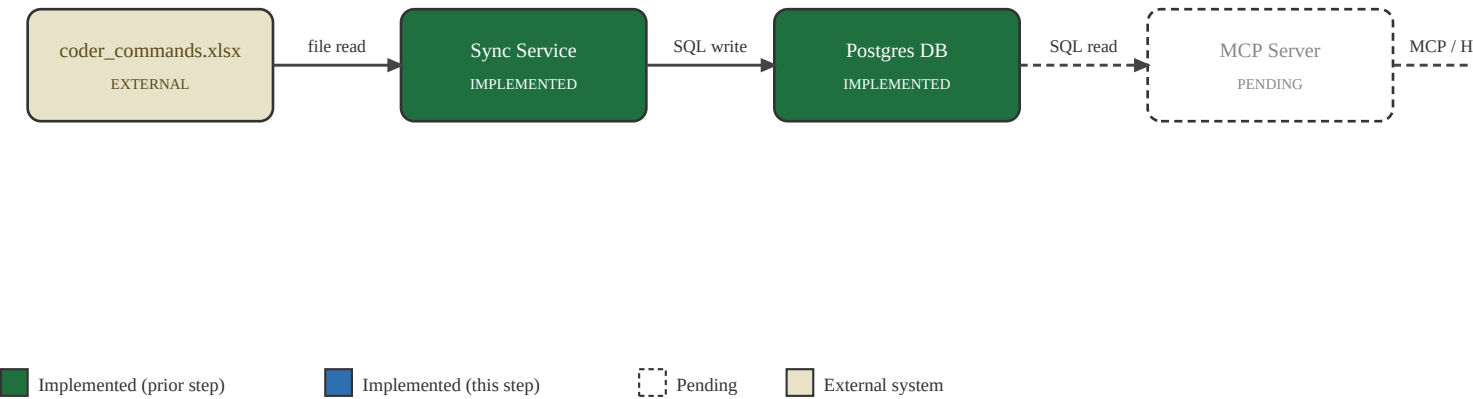
DOCUMENT	CoderCommands MCP System — Step Record
SYSTEM	CoderCommands MCP Command-Reference Server
STEP	04 — Sync Service (Watcher + Transactional Write)
DATE	2026-08-15
REVISION	1.0
STATUS	Complete

1. Purpose & Scope

Turn the pure ingest function from Step 3 into a running, unattended service: detect edits to the source workbook without a manual trigger, and commit them into Postgres safely. This is the closed-loop element of the system -- without it, Postgres is a one-time snapshot, not a live mirror of the workbook.

2. System Context Diagram

Signal-flow view of the four functional units (Sync Service, Postgres, MCP Server, FastAPI Gateway) plus the two external endpoints (source workbook, Claude Code client). Fill/border indicates build state as of this step.



3. File / Component Register

Role of every file or component introduced or modified in this step, with its upstream and downstream interfaces (I/O-list style).

Ref	File / Component	Role	Interface In	Interface Out
R1	sync/watcher.py	Polls the xlsx modified-time every SYNC_POLL_SECONDS; on change, calls ingest.normalize_workbook() and performs a per-topic DELETE+INSERT inside one transaction, then logs the pass to sync_runs.	coder_commands.xlsx, ingest.py	Postgres (commands, sync_runs)
R2	sync/requirements.txt	Pins openpyxl (xlsx parsing) and psycopg[binary] (Postgres driver) for the container build.	—	—
R3	sync/Dockerfile	python:3.12-slim base; installs requirements.txt, copies ingest.py + watcher.py, runs watcher.py as the container's foreground process.	—	Docker Compose (Step 7)

4. Work Performed

- Poll loop compares `os.path.getmtime()` against the last-seen value every SYNC_POLL_SECONDS (default 7s) rather than relying on filesystem change events, per the Step 3 note that Docker Desktop bind-mount events are unreliable on Windows.
- Each sync pass: DELETE FROM commands WHERE topic=%s then re-insert, per topic, inside one `conn.transaction()` block -- an edit, reorder, or deletion in any sheet is handled identically, with no row-level diffing logic to maintain.
- Every pass writes a row to `sync_runs` (start/finish time, source file mtime, topics/rows synced, status, error) so sync health is queryable, not just visible in container logs.
- Added a bounded connect-retry loop on startup (10 attempts, 2s apart) so the container doesn't crash-loop if it starts slightly before Postgres is accepting connections.
- Validated the full pipeline end-to-end against a disposable Postgres instance and the real `coder_commands.xlsx` before wiring it into the compose stack in Step 7.

5. Issues Encountered & Resolution

Issue	Root Cause	Resolution
Publishing a throwaway Postgres container's port to the host for testing failed on the first four ports tried (55432, 25432, 35432, 45432): "bind: An attempt was made to access a socket in a way forbidden by its access permissions."	Those ports fall inside a range Windows has excluded for dynamic/Hyper-V use, which Docker Desktop cannot bind to regardless of what else is listening.	Used port 15432 instead, which bound cleanly. Noted for Step 7: any host port mapping added later should be picked with this exclusion range in mind, or left unpublished entirely as planned.

Issue	Root Cause	Resolution
A second test run threw <code>FileNotFoundError</code> on the exact same <code>xlsx</code> path string that had worked moments earlier in a different command.	The failing case embedded the POSIX-style path (<code>/c/Users/...</code>) as a literal string inside a Python script passed via <code>`python -c "...".`</code> . Git Bash's MSYS layer only rewrites POSIX-looking paths that appear as actual shell command-line arguments or <code>env-var-prefix</code> assignments -- not text sitting inside a quoted script body, even when that text originated from a shell variable. Windows-native Python then tried to resolve <code>/c/Users/...</code> literally and failed.	Used <code>`cygpath -w`</code> to convert to a genuine Windows path (<code>C:\Users\...</code>) before embedding it in the Python source. This is distinct from the Step 2 issue (a docker volume-mount argument, fixed with <code>MSYS_NO_PATHCONV</code>) -- same root cause, different fix depending on where the path is consumed.

6. Verification Record

Check	Method	Result
Initial sync against the real workbook matches Step 3's standalone ingest count	608 rows / 14 topics via <code>sync_once()</code> against a disposable Postgres instance	PASS
Re-sync on an unchanged file does not duplicate rows	Ran <code>sync_once()</code> twice; row count stayed at 608, <code>sync_runs</code> gained one new audit row	PASS
Deletions in the source propagate correctly (no orphaned rows)	Removed the last row of the 'ai' sheet in a scratch copy of the workbook, re-synced: ai 7→6 rows, total 608→607	PASS
<code>sync_runs</code> accurately logs each pass	Queried <code>sync_runs</code> after each of the above runs	PASS
Startup connect-retry against a not-yet-ready Postgres	Not exercised in this pass -- logic reviewed by inspection only	N/A (deferred to Step 8)

7. Next Step / Open Dependencies

Step 5 builds the MCP server that reads this Postgres data via `search_commands`, `list_topics`, and per-topic browse tools, over Streamable HTTP.